

Subiect 06 – Platformă de recrutare

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire_pattern*, *cts.nume.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată unei platforme de recrutare

3p. O platformă de recrutare primește CV-uri în formate diferite de la mai mulți furnizori externi. Aplicația internă poate procesa doar candidați într-un format standard, cu metode pentru obținerea numelui, experienței și competențelor. Se cere integrarea unui CV extern într-un flux deja existent de evaluare, fără modificarea clasei externe și fără schimbarea codului care procesează candidații interni. Pentru implementare se va folosi interfața *AbstractCandidatPlatforma*.

1.5p. Să se testeze soluția prin:

- procesarea unui candidat intern;
- procesarea unui candidat provenit dintr-un sistem extern;
- folosirea aceluiași evaluator pentru ambele cazuri;
- evidențierea clasei externe care este integrată în formatul platformei.

3p. În procesul de recrutare, aplicația trebuie să filtreze succesiv candidații după mai multe criterii: experiență minimă, existența unor competențe obligatorii, disponibilitate pentru interviu și compatibilitate salarială. Fiecare filtru decide dacă un candidat este respins sau transmis către următorul filtru. Ordinea filtrelor trebuie să poată fi schimbată ușor. Pentru implementare se va folosi interfața *AbstractFiltruCandidat*.

1.5p. Să se testeze soluția prin:

- definirea a cel puțin 3 filtre succesive;
- testarea unor candidați acceptați și respinși;
- schimbarea ordinii filtrelor;
- afișarea filtrului la care candidatul este respins.

```
public interface AbstractCandidatPlatforma {  
    String obtineNume();  
    int obtineAniExperienta();  
    String[] obtineCompetente();  
}
```

```
public abstract class AbstractFiltruCandidat {  
    protected AbstractFiltruCandidat urmator;
```

```
    public void seteazaUrmator(AbstractFiltruCandidat urmator) {  
        this.urmator = urmator;  
    }
```

```
    public abstract boolean proceseaza(Candidat candidat);  
}
```